

UZF 421 E - Attitude Determination and Control

Homework Assignment 4

Ahmet Fevzi Akargöl
Student ID: 110200170

May 4, 2025

Task 1: Gyroscope Sensor Research

Search for three different gyroscopes used on nanosatellites (you may find them as part of the inertial measurement unit -IMU component). Fill the Table 1 for the related sensors' characteristics. At least 5 of the items need to be filled for each sensor.

Table 1: Characteristics of the gyroscope samples.

	Sensor A (Epson M-G370PDT)	Sensor B (Honeywell HG4930AA51)	Sensor C (Sensoror STIM377H)
Website of the Datasheet	Link	Link	Link
Sensor Type	MEMS IMU (6-DOF)	MEMS IMU (6-DOF)	MEMS IMU (6-DOF + Incl.)
Size in m	$0.024 \times 0.024 \times 0.01$	$0.065 \times 0.051 \times 0.0355$	Volume $< 3.5 \times 10^{-5} \text{ m}^3$
Mass in kg	0.01	0.14	< 0.055
Operating Temperature in °C	-40 to +85	-54 to +85	-40 to +85
Range in deg/s	± 200	± 400	± 400
Bias Stability in deg/h	0.8 (typ)	0.45 (1σ)	0.3 (typ)
Noise Density in deg/s/ $\sqrt{\text{Hz}}$	0.0005 (from ARW)	0.001 (from ARW)	0.0025 (from ARW)
Frequency in Hz	189 (-3dB BW)	180 (-3dB BW)	-

Note: Noise Density

calculated from Angular Random Walk (ARW) using Noise Density = ARW / 60. Refer to datasheets for exact specifications.

Task 3: Gyroscope Bias Simulation

The gyroscope bias vector (b_{rg}) evolution over time was simulated using Euler's method applied to the random walk model $\dot{b}_{rg} = v_{rgb}$, with initial bias $b_{rg}(0)$ and noise standard deviation σ_{rgb} specific to n=0 (after unit conversion to rad/s and rad/s²). The components of the resulting bias vector are plotted below over N=6000 samples.

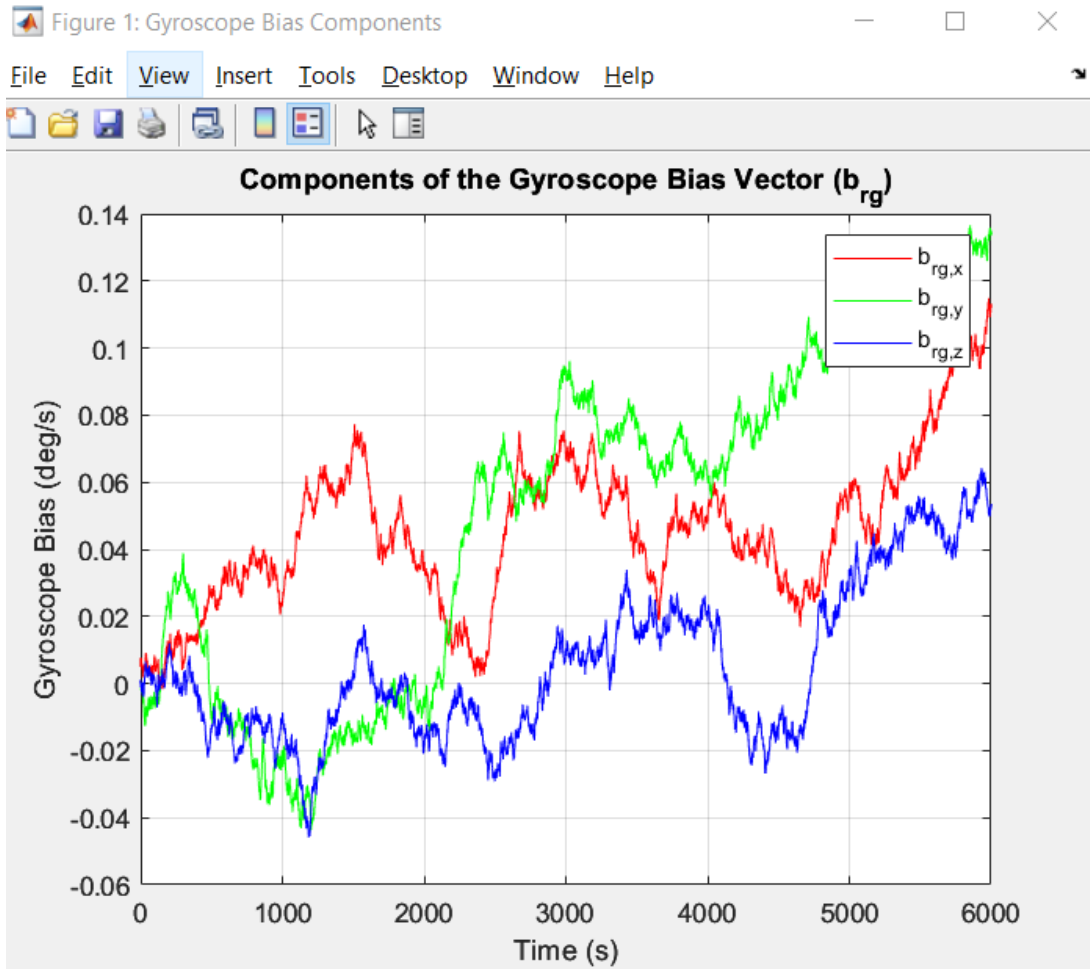


Figure 1: Components of the Gyroscope Bias Vector (b_{rg}) vs Time.

Task 4: Gyroscope Measurement Simulation Plots

Gyroscope measurements ($\tilde{\omega}_{BN}$) were simulated for N=6000 samples using the equation $\tilde{\omega}_{BN} = \omega_{BN}^{true} + b_{rg} + v_{rg}$. Here, ω_{BN}^{true} is the true angular velocity from HW1 (n=0), b_{rg} is the simulated time-varying bias from Task 3, and v_{rg} is the measurement noise (σ_{rg} specific to n=0, converted to rad/s). The following plots show the components for 3 cases:

- $\tilde{\omega}_{BN}^{Case\ 1} = \omega_{BN}^{true} + b_{rg} + v_{rg}$
- $\tilde{\omega}_{BN}^{Case\ 2} = \omega_{BN}^{true} + v_{rg}$
- $\tilde{\omega}_{BN}^{Case\ 3} = \omega_{BN}^{true}$

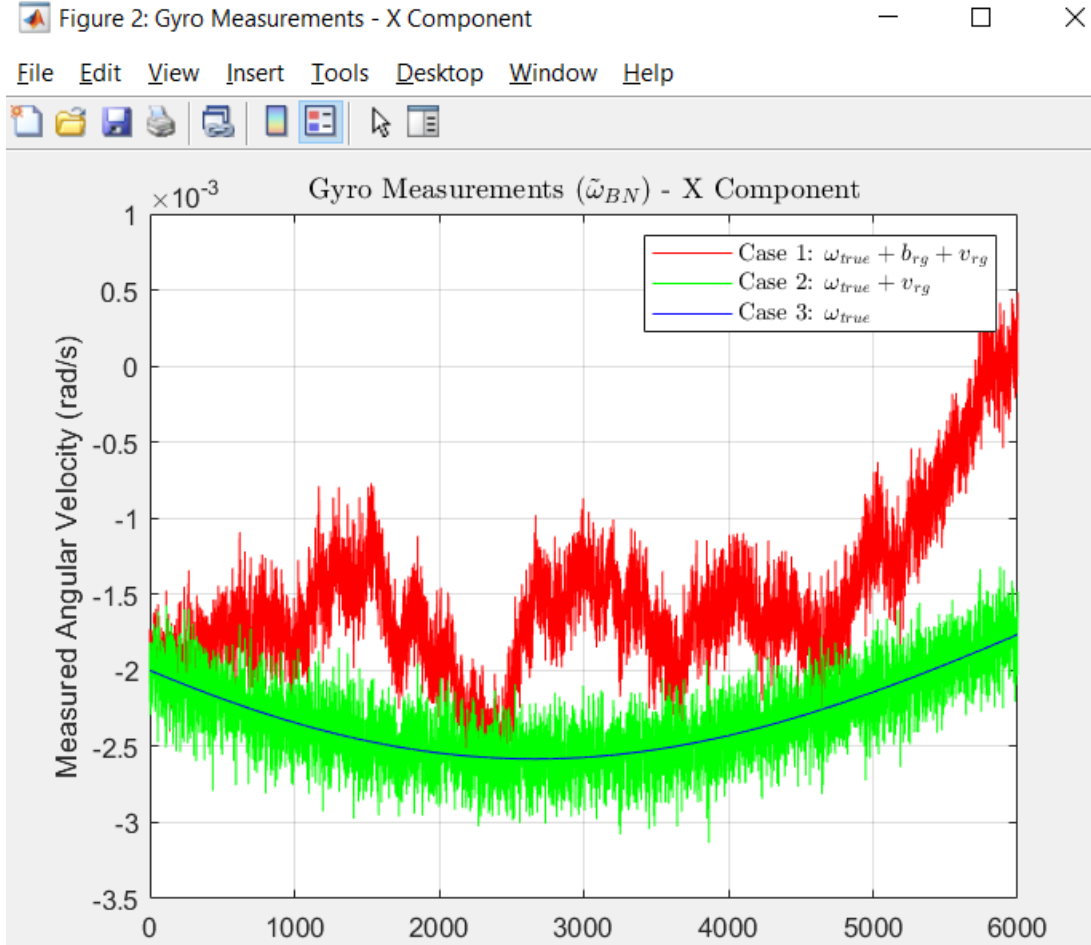


Figure 2: Gyroscope Measurement ($\tilde{\omega}_{BN}$) for X-Component (Cases 1, 2, 3).

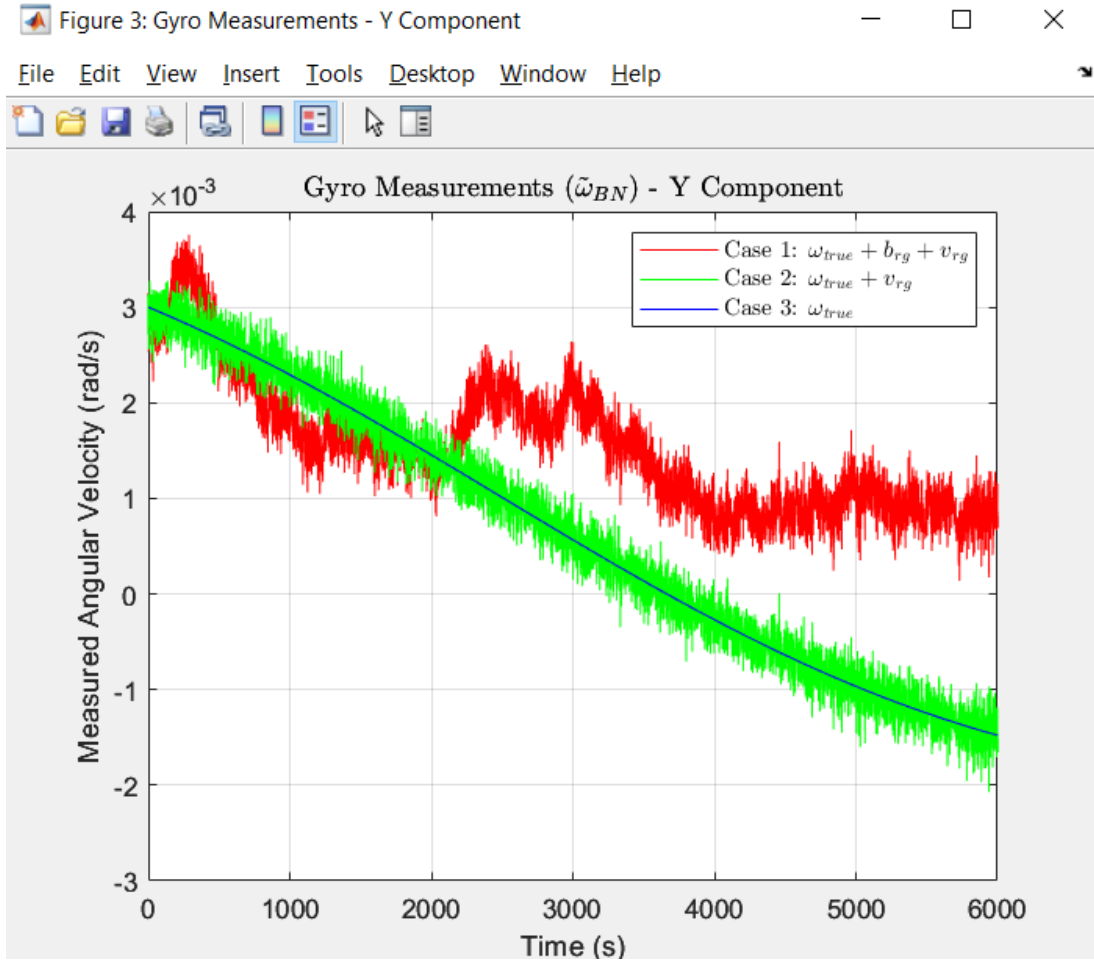


Figure 3: Gyroscope Measurement ($\tilde{\omega}_{BN}$) for Y-Component (Cases 1, 2, 3).

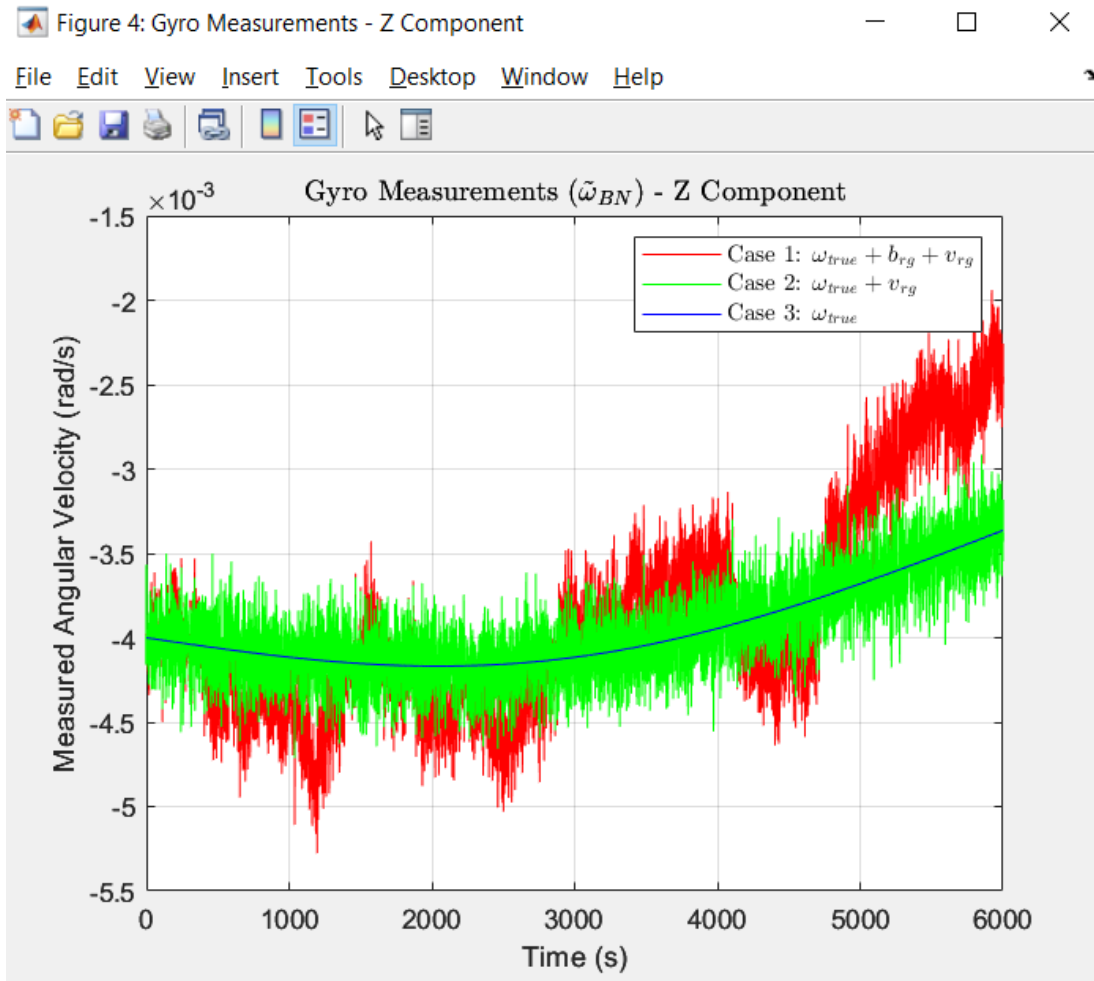


Figure 4: Gyroscope Measurement ($\tilde{\omega}_{BN}$) for Z-Component (Cases 1, 2, 3).

Task 5: Explanation for Bias and Noise

What is the reason of adding bias or noise terms on top of the true angular velocity vector?

Real-world gyroscope sensors we use are unfortunately not perfect. They make various errors when measuring the satellite's true angular velocity (ω_{BN}^{true}). That's why we need to include these errors in our simulations to make our results more realistic.

- **Bias** (b_{rg}): We can think of this as a systematic error the sensor constantly makes. It's like the gyroscope incorrectly reports a slight rotation even when it's perfectly still, or this error slowly changes over time (drift). It can originate from manufacturing imperfections or external factors like temperature. By adding bias to the model, we mimic this constant or slowly changing error.
- **Noise** (v_{rg}): These are instantaneous, random, small deviations during the measurement. They can be caused by factors like interference in electronic circuits or vibrations. By adding noise, we model this randomness and uncertainty in the measurements.

In short, the reason we add bias and noise is to ensure that the simulation reflects the behavior of a **real sensor**, not an ideal one. This way, we can better test how algorithms that will use this sensor data (like algorithms determining the satellite's orientation) will cope with flawed measurements in the real world.

A Appendix: MATLAB Code

The MATLAB code used for Homework 4 is provided below. The code uses initial conditions for $n=0$ and calculates the true angular velocity ω_{BN}^{true} based on Homework 1.

```
1  clc; clear; close all;
2
3  %% Constants definitions
4  n = 0; % Last digit of student number
5  N = 6000; % Number of samples
6  delta_t = 1; % Sampling time (seconds)
7  time = (0:N)*delta_t; % Time vector from 0 to N seconds
8
9  % Conversion factor
10 deg2rad_val = pi/180;
11 rad2deg_val = 180/pi;
12
13 %
14 % -----
15 % SECTION FROM HOMEWORK 1 (for n=0) - Calculating True Angular Velocity
16 % -----
17
18 % Initial attitude angles (rad) for n=0
19 psi_0 = -0.005 - 0.002*n; % Yaw
20 theta_0 = 0.01 - 0.002*n; % Pitch
21 phi_0 = -0.01 - 0.002*n; % Roll
22
23 % Initial angular velocities (rad/s) for n=0
24 w1_0 = -0.002 - 0.0001*n;
25 w2_0 = 0.003 - 0.0001*n;
26 w3_0 = -0.004 - 0.0001*n;
27
28 % Mass moments of inertia (kg.m^2)
29 I1 = 2.1e-3;
30 I2 = 2.0e-3;
31 I3 = 1.9e-3;
32
33 % Disturbance torques (N.m)
34 L1 = 3.6e-10;
35 L2 = 3.6e-10;
36 L3 = 3.6e-10;
37
38 % --- Propagate Angular Velocities (Euler Method) ---
39 w = [w1_0, w2_0, w3_0]'; % Use column vector
40 w_true_history = zeros(3, N+1); % Store true angular velocity (rad/s)
41 w_true_history(:, 1) = w;
42
43 for i = 1:N
44     w1 = w(1); w2 = w(2); w3 = w(3);
45     dw1 = (-(I3 - I2)*w2*w3 + L1) / I1;
46     dw2 = (-(I1 - I3)*w3*w1 + L2) / I2;
47     dw3 = (-(I2 - I1)*w1*w2 + L3) / I3;
48
49     % Euler integration step for omega
50     w = w + [dw1; dw2; dw3] * delta_t;
51     w_true_history(:, i+1) = w;
```

```

51 end
52
53 % w_true_history now contains omega_BN_true in rad/s for N+1 time steps
54
55 %
    -----
56 % HOMEWORK 4 CALCULATIONS (for n=0)
57 %
    -----
58
59 % --- Define Constants for HW4 with Unit Conversion ---
60 % Initial bias in deg/s
61 b_rg_0_deg = [5; 1+n; 1-n] * 1e-3; % deg/s
62 % Convert initial bias to rad/s
63 b_rg_0_rad = b_rg_0_deg * deg2rad_val; % rad/s
64
65 % Measurement noise standard deviation in deg/s
66 sigma_rg_deg = 1e-2; % deg/s
67 % Convert measurement noise std dev to rad/s
68 sigma_rg_rad = sigma_rg_deg * deg2rad_val; % rad/s
69
70 % Bias random walk noise standard deviation in deg/s^2
71 sigma_rgb_deg = 1e-3; % deg/s^2
72 % Convert bias random walk noise std dev to rad/s^2
73 sigma_rgb_rad = sigma_rgb_deg * deg2rad_val; % rad/s^2
74
75 % --- Simulate Gyro Bias (Random Walk) using Euler Method (Task 3) ---
76 b_rg_history_rad = zeros(3, N+1); % History of bias vector in rad/s
77 b_rg_history_rad(:, 1) = b_rg_0_rad; % Set initial bias
78
79 for k = 1:N
80     % Generate bias random walk noise for this step (rad/s^2)
81     v_rgb_k = sigma_rgb_rad * randn(3,1);
82     % Euler integration for bias
83     b_rg_history_rad(:, k+1) = b_rg_history_rad(:, k) + v_rgb_k *
        delta_t;
84 end
85
86 % --- Plot Gyro Bias Components (Task 3) ---
87 figure('Name', 'Gyroscope Bias Components');
88 plot(time, b_rg_history_rad(1,:) * rad2deg_val, 'r', 'DisplayName', 'b_{
    rg,x}');
89 hold on;
90 plot(time, b_rg_history_rad(2,:) * rad2deg_val, 'g', 'DisplayName', 'b_{
    rg,y}');
91 plot(time, b_rg_history_rad(3,:) * rad2deg_val, 'b', 'DisplayName', 'b_{
    rg,z}');
92 hold off;
93 xlabel('Time (s)');
94 ylabel('Gyroscope Bias (deg/s)');
95 title('Components of the Gyroscope Bias Vector (b_{rg})'); % No special
    chars here
96 legend;
97 grid on;
98
99 % --- Simulate Gyro Measurements (Task 4) ---

```

```

100 w_meas_case1 = zeros(3, N+1); % Case 1: omega_true + b_rg + v_rg
101 w_meas_case2 = zeros(3, N+1); % Case 2: omega_true + v_rg
102 w_meas_case3 = zeros(3, N+1); % Case 3: omega_true
103
104 for k = 1:(N+1)
105     % Generate measurement noise for this step (rad/s)
106     v_rg_k = sigma_rg_rad * randn(3,1);
107
108     % True angular velocity (rad/s)
109     omega_true_k = w_true_history(:, k);
110
111     % Bias at this step (rad/s)
112     b_rg_k = b_rg_history_rad(:, k);
113
114     % Calculate measurements for the three cases (all in rad/s)
115     w_meas_case3(:, k) = omega_true_k;
116     w_meas_case2(:, k) = omega_true_k + v_rg_k;
117     w_meas_case1(:, k) = omega_true_k + b_rg_k + v_rg_k;
118 end
119
120 % --- Plot Gyro Measurements (Task 4) ---
121 % Plot units should be rad/s
122
123 figure('Name', 'Gyro Measurements - X Component');
124 plot(time, w_meas_case1(1,:), 'r', 'DisplayName', 'Case 1:  $\tilde{\omega}_{BN}$ 
    +  $b_{rg}$  +  $v_{rg}$ '); % Use $ for LaTeX math
125 hold on;
126 plot(time, w_meas_case2(1,:), 'g', 'DisplayName', 'Case 2:  $\tilde{\omega}_{BN}$ 
    +  $v_{rg}$ '); % Use $ for LaTeX math
127 plot(time, w_meas_case3(1,:), 'b', 'DisplayName', 'Case 3:  $\tilde{\omega}_{BN}$ 
    '); % Use $ for LaTeX math
128 hold off;
129 xlabel('Time (s)');
130 ylabel('Measured Angular Velocity (rad/s)');
131 % Correction: Added 'Interpreter', 'latex' and used $ for math mode
132 title('Gyro Measurements ( $\tilde{\omega}_{BN}$ ) - X Component', 'Interpreter', 'latex');
133 legend('Interpreter', 'latex'); % Also set interpreter for legend
134 grid on;
135
136 figure('Name', 'Gyro Measurements - Y Component');
137 plot(time, w_meas_case1(2,:), 'r', 'DisplayName', 'Case 1:  $\tilde{\omega}_{BN}$ 
    +  $b_{rg}$  +  $v_{rg}$ '); % Use $ for LaTeX math
138 hold on;
139 plot(time, w_meas_case2(2,:), 'g', 'DisplayName', 'Case 2:  $\tilde{\omega}_{BN}$ 
    +  $v_{rg}$ '); % Use $ for LaTeX math
140 plot(time, w_meas_case3(2,:), 'b', 'DisplayName', 'Case 3:  $\tilde{\omega}_{BN}$ 
    '); % Use $ for LaTeX math
141 hold off;
142 xlabel('Time (s)');
143 ylabel('Measured Angular Velocity (rad/s)');
144 % Correction: Added 'Interpreter', 'latex' and used $ for math mode
145 title('Gyro Measurements ( $\tilde{\omega}_{BN}$ ) - Y Component', 'Interpreter', 'latex');
146 legend('Interpreter', 'latex'); % Also set interpreter for legend
147 grid on;
148
149 figure('Name', 'Gyro Measurements - Z Component');

```

```

150 plot(time, w_meas_case1(3,:), 'r', 'DisplayName', 'Case 1:  $\omega_{BN}$ 
       $^{\text{true}} + b_{\text{rg}} + v_{\text{rg}}$ '); % Use $ for LaTeX math
151 hold on;
152 plot(time, w_meas_case2(3,:), 'g', 'DisplayName', 'Case 2:  $\omega_{BN}$ 
       $^{\text{true}} + v_{\text{rg}}$ '); % Use $ for LaTeX math
153 plot(time, w_meas_case3(3,:), 'b', 'DisplayName', 'Case 3:  $\omega_{BN}$ 
       $^{\text{true}}$ '); % Use $ for LaTeX math
154 hold off;
155 xlabel('Time (s)');
156 ylabel('Measured Angular Velocity (rad/s)');
157 % Correction: Added 'Interpreter', 'latex' and used $ for math mode
158 title('Gyro Measurements ( $\tilde{\omega}_{BN}$ ) - Z Component', '
      Interpreter', 'latex');
159 legend('Interpreter', 'latex'); % Also set interpreter for legend
160 grid on;
161
162 %
      -----
163 % END OF HOMEWORK 4 SIMULATION
164 %
      -----

```

Listing 1: MATLAB Code for Homework 4 Simulation (n=0)